

JS



JavaScript

Compreender o DOM é entender como o JavaScript interage diretamente com os elementos da página, transformando a estrutura em comportamento.



Por que aprender JavaScript?

- JavaScript é a **linguagem do navegador**.
- Está presente em tudo: jogos online, redes sociais, aplicativos mobile, até carros inteligentes.
- É **porta de entrada** para outras áreas (back-end com Node.js, apps móveis com React Native, até IoT).



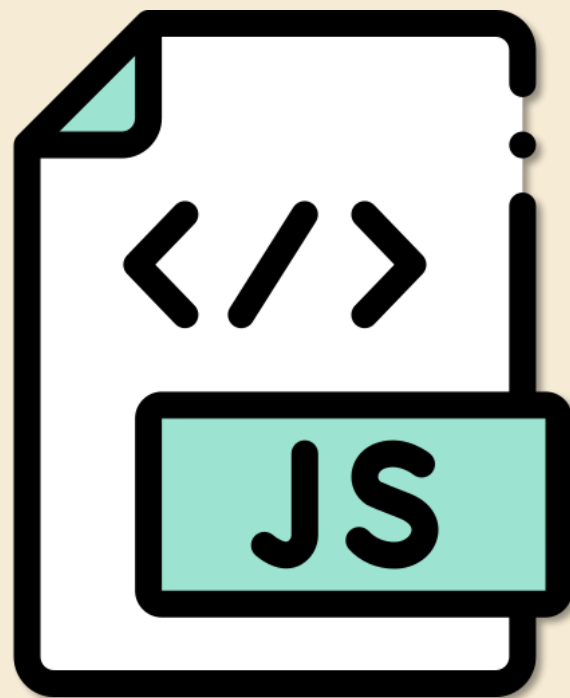
Onde o JavaScript roda?

- **Navegadores:** Chrome (motor V8), Firefox (SpiderMonkey), Safari, Edge.
- **Fora do navegador:** Node.js (servidores), React Native (apps móveis), Electron (apps de desktop).
- **Exemplo real:** Slack e VS Code são feitos com JavaScript.



Relembrando Lógica de Programação

- Algoritmo = receita de bolo.

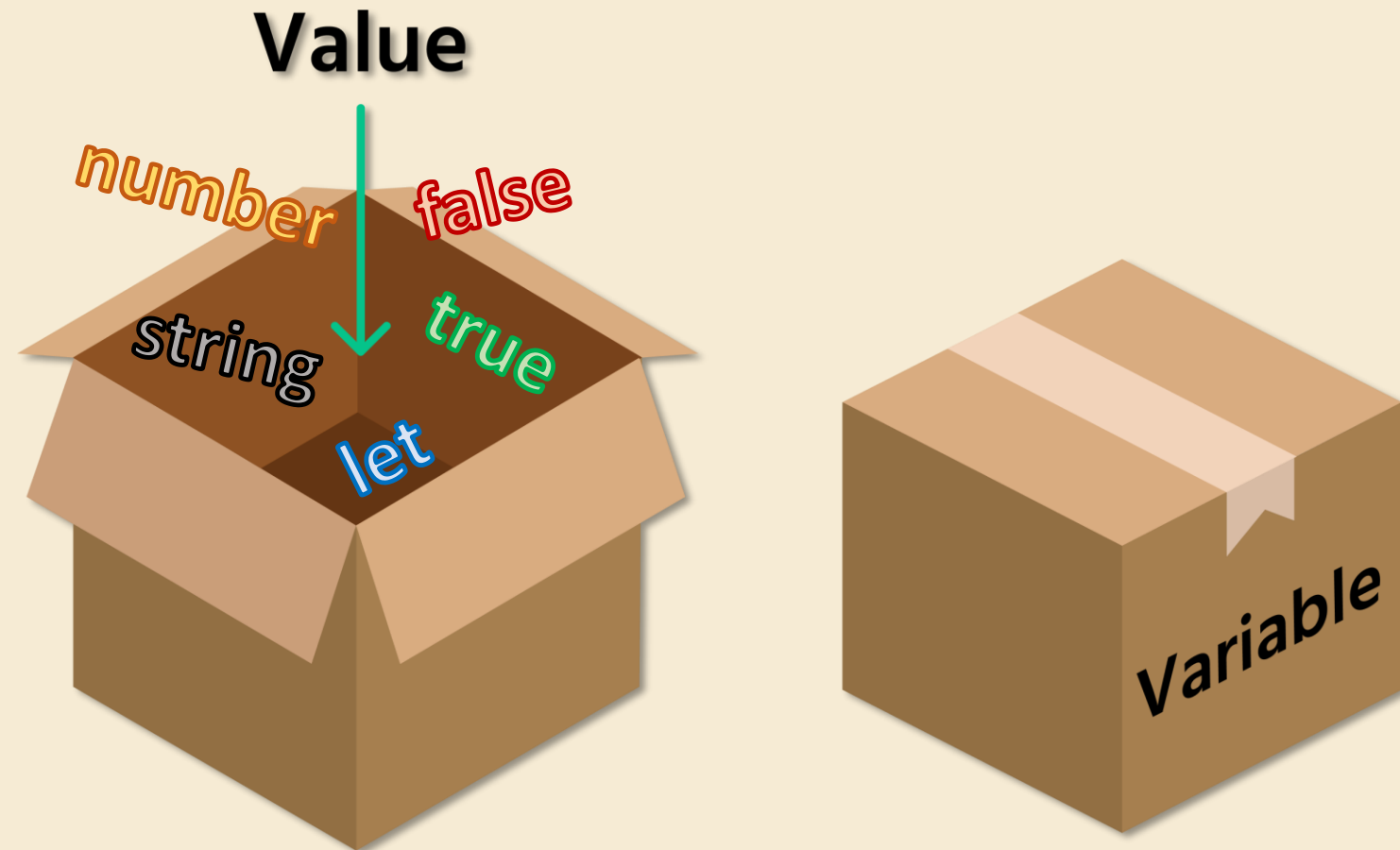


=



Relembrando Lógica de Programação

- Variáveis = caixas onde guardamos informações.



Relembrando Lógica de Programação

- Tipos de dados = string (texto),
number (números), boolean (verdadeiro/falso).

Hello, World!

true

2025

false



Relembrando Lógica de Programação

- Laços de repetição = “Mexer a sopa na panela”.



Mexer até que a sopa esteja homogênea.



DOM (Document Object Model) — a planta interativa da sua página

Hoje vamos pensar o HTML como uma casa — o DOM é a planta dessa casa, em formato de árvore. O JavaScript é a chave que nos permite mexer nos cômodos, adicionar móveis, trocar lâmpadas — em outras palavras, alterar a página enquanto o usuário interage com ela.

Objetivos da aula

- Entender o que é DOM e por que ele existe
- Ver a estrutura em árvore (nós)
- Aprender as formas básicas de acessar e manipular elementos com JavaScript
- Fazer um mini-demo: "acender/apagar a lâmpada"
- Atividade prática: mini To-Do interativo

Ao final desta apresentação o aluno deverá conseguir inspecionar uma página, localizar elementos no DOM, e realizar mudanças simples com JavaScript.



O que é DOM? (metáfora)

1

Casa (site)

Sua página web é como uma casa construída

2

Planta (DOM)

DOM = *Document Object Model* → representação programática do HTML

3

Ferramentas (JavaScript)

JavaScript usa essa planta para agir sobre os móveis (elementos)

Imagine que você tem a planta da casa à sua frente: está escrito onde está cada cômodo e cada móvel. O DOM é essa planta para o navegador — e o JavaScript usa essa planta para agir sobre os móveis (elementos).



Estrutura: a árvore do DOM

O DOM é uma **árvore** formada por **nós** (nodes).

- Cada elemento HTML é um *element node*
- O texto dentro de um elemento é um *text node*
- Atributos são nós associados

Exemplo simplificado:

<html> → <body> → <div> → <p> → texto

Pense em cada nó como um nó em uma árvore genealógica: podemos subir (parent), descer (children) e navegar entre irmãos (nextSibling / previousSibling).



Tipos de nós importantes

Element node
ex.: `<div>`, `<p>`

Text node
conteúdo textual

Attribute node
ex.: `class="ativo"`

Comment node
`<!-- comentário -->`

Saber distinguir nós evita confusões quando manipulamos `childNodes` vs `children` e quando alteramos `textContent` vs `innerHTML`.



Como acessar elementos (seletores)

- `document.getElementById('id')` — rápido, único
- `document.getElementsByClassName('classe')` — HTMLCollection (vivo)
- `document.getElementsByTagName('tag')` — lista de tags
- `document.querySelector('.classe')` — primeiro que combina (CSS selector)
- `document.querySelectorAll('li.item')` — NodeList estático

Código de exemplo:

```
const el1 = document.getElementById('titulo');const primeiroBotao =  
document.querySelector('button');const itens =  
document.querySelectorAll('.item');
```



Manipulação básica: conteúdo, atributos e classes

textContent

altera texto puro

innerHTML

altera HTML (cuidado: XSS)

setAttribute / getAttribute

para atributos

classList.add/remove/toggle

para classes

Exemplos curtos:

```
element.textContent = 'Olá, turma!';element.innerHTML = '<strong>Olá</strong>'; // cuidado com HTML vindo do usuário  
element.setAttribute('data-id', '123');element.classList.toggle('ativo');
```



Criar e remover elementos

- `document.createElement()` cria
- `appendChild`, `insertBefore`, `append` para inserir
- `remove()` para excluir
- `DocumentFragment` para melhorar performance em inserções em lote

Exemplo:

```
const li = document.createElement('li'); li.textContent = 'Tarefa nova'; document.querySelector('#lista').appendChild(li); // removerli.remove(); // inserir em lote (performance)const frag = document.createDocumentFragment(); for (let i=0; i<100; i++){ const x = document.createElement('li'); x.textContent = 'Item ' + (i+1); frag.appendChild(x); } document.querySelector('#lista').appendChild(frag);
```

Eventos: a ponte para a interação

- Eventos comuns: click, input, submit, keydown
- Adicionar ouvintes: `element.addEventListener('evento', callback)`
- Delegação de eventos (útil para listas dinâmicas)

Exemplo (delegação):

```
document.querySelector('#lista').addEventListener('click', (e) => { if (e.target.tagName === 'LI') { e.target.classList.toggle('completa'); } });
```

